Simulation of a Cold-Chain Logistics Monitoring System Using OMNeT++, INET, and FLoRa

Ali Murat Chatkha, Irem Dede
Computer Engineering Program
Middle East Technical University Northern Cyprus Campus
CNG 476 - System Simulation
Student IDs: 2486843, 2584910
Instructor: Asst. Prof. Dr. Muhammad Toaha R. Khan
Semester: Spring 2025-2026
GitHub: https://github.com/irem0410/COLDCHAIN-MONITORING.git
Date of Submission: 18 May 2026

***Abstract*—Cold-chain logistics systems are used for transporting temperature-sensitive products such as fresh food, medicine, vaccines, and laboratory materials. If environmental conditions inside the container are not monitored correctly, the quality of the transported product can be damaged. Real life testing of temperature and humidity variations would be costly, thus, simulation-based tests are necessary for monitoring environmental variations. In this project, a simulation-based cold-chain monitoring system was implemented using OMNeT++, INET, and FLoRa. The implemented system contains IoT sensor nodes, a LoRa gateway, and a backend monitoring server.**

The simulation was modelled as a discrete-event simulation. Sensor nodes periodically generate environmental readings and transmit them through LoRaWAN communication. Random number generation was used to represent stochastic temperature and humidity readings. Repeated Monte Carlo simulation runs were executed in order to observe the effect of randomness on packet delivery, collisions, alarm generation, and energy consumption.

The results show that the system can deliver most of the generated sensor packets successfully, with an average Packet Delivery Ratio of approximately 81.64%. However, packet collisions were observed because of wireless contention in the LoRa channel. The project shows that discrete-event simulation can be useful for analysing IoT-based cold-chain monitoring systems before real implementation.

***Index Terms*—System simulation, OMNeT++, INET, FLoRa, LoRaWAN, cold-chain logistics, IoT, discrete-event simulation, Monte Carlo simulation.**

I. Introduction

Cold-chain logistics is an important part of transporting sensitive products such as vaccines, medicine, frozen food, fresh fruit, and other perishable goods. These products must stay within a safe temperature and humidity range during transportation. Maintaining stable environmental conditions during transportation is critical for preserving the quality and safety of temperature-sensitive products in cold-chain logistics systems [13]. If the container temperature becomes too high or if the humidity level changes too much, the transported product may become unsuitable for use.

Because of this, modern cold-chain systems can use IoT sensor nodes inside refrigerated containers. Recent cold-chain systems increasingly utilize IoT sensors for real-time monitoring and optimization of storage and transportation conditions [14]. IoT enables continuous and end-to-end monitoring of environmental parameters such as temperature and humidity, helping reduce spoilage risks during transportation [2]. These sensors periodically measure environmental values and send the information to a monitoring center. Continuous monitoring approaches provide cumulative environmental information throughout transportation and storage operations [15]. If the value exceeds a threshold, an alarm can be generated. However, testing this type of system directly in real life can be expensive and difficult. Therefore, simulation is a practical way to test the system before real deployment.

This project implements a cold-chain logistics monitoring system using OMNeT++, INET, and FLoRa. OMNeT++ is used as the main discrete-event simulation environment. FLoRa is used for LoRaWAN communication modelling, and INET is used for backend network components. LoRaWAN is widely used in IoT and wireless sensor network applications due to its long-range communication capability and low power consumption characteristics [6]. The system contains three sensor nodes, one LoRa gateway, and one backend server.

The main purpose of the project is to evaluate the reliability of the communication system and the behaviour of sensor readings under stochastic conditions. The project focuses on packet delivery ratio, collision count, alarm count, energy consumption, and queue behaviour.

II. Selected Project Description

Time-series-based stochastic modelling is commonly used in IoT environments to simulate realistic environmental fluctuation and sensor behaviour [5]. The selected project is a cold-chain logistics monitoring system. The real-world system being modelled is a refrigerated transportation environment where containers are equipped with IoT sensors. These sensor nodes periodically collect temperature and humidity readings and send them to a backend monitoring server through a LoRaWAN gateway. Wireless sensor network applications de-

pend on reliable and energy-efficient communication between sensor nodes and the monitoring infrastructure [1].

The simulation is useful because the performance of such a system depends on both environmental uncertainty and wireless communication limitations. Temperature and humidity are among the most critical environmental parameters affecting the quality and shelf life of perishable products [14]. Nature of the project suggests sensor readings are not fixed values, and wireless packet delivery may be affected by collisions, interference, and packet loss.

Table I summarizes the selected project.

TABLE I
SELECTED PROJECT SUMMARY

Item	Description
Selected Project Title	Cold-Chain Logistics Monitoring System
Application Domain	IoT-based logistics and environmental monitoring
Main Problem	Monitoring sensitive goods and transmitting sensor data reliably
Simulation Objective	Evaluate LoRaWAN packet delivery, collisions, alarms, and energy usage
Main Users/Entities	Sensor nodes, LoRa gateway, backend server, logistics operator
Main Performance Metrics	PDR, collisions, alarm count, energy consumption, queue length

III. SYSTEM ARCHITECTURE

The implemented system architecture contains three main parts. The first part is the sensor node layer. Sensor nodes represent IoT devices placed inside refrigerated containers. They generate temperature and humidity values and transmit packets periodically. Periodic and query-driven reporting models are commonly used in wireless sensor network applications depending on monitoring requirements and energy constraints [1].

The second part is the LoRa gateway. The gateway receives wireless packets from sensor nodes. Since multiple nodes may transmit in the same wireless environment, packet collisions can occur at the gateway.

The third part is the backend monitoring server. The backend server receives the forwarded information and represents the monitoring side of the cold-chain system.

Fig. 1 shows the implemented topology in OMNeT++.

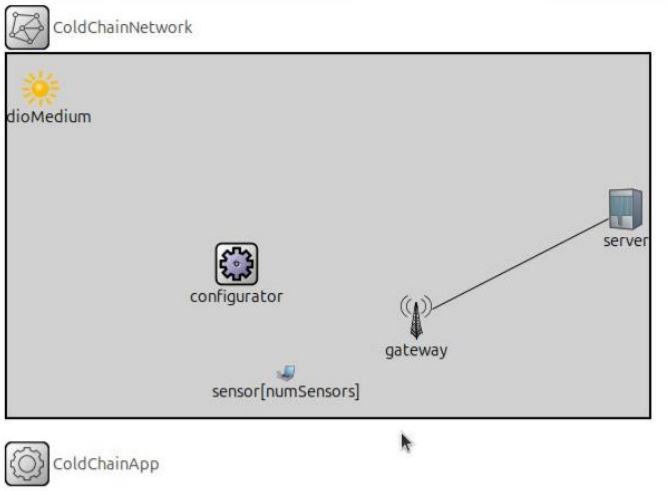

Fig. 1. Implemented cold-chain monitoring topology in OMNeT++.

The simulation topology includes three sensor nodes, one LoRa gateway, and one backend server. The sensor nodes communicate with the gateway over LoRaWAN. The backend part is represented with INET-based network components.

IV. SYSTEM MODEL AND ASSUMPTIONS

The system was modelled as a discrete-event simulation. The state of the system changes only when an event occurs. OMNeT++ follows the discrete-event simulation approach where network behaviour is modeled through timestamped message-based events [9]. Examples of events are packet generation, packet transmission, packet reception, collision detection, and alarm generation.

TABLE II
ENVIRONMENTAL INPUT PARAMETERS USED IN THE SIMULATION

Parameter	Value	Purpose
Temperature mean, μ_T	0 °C	Target refrigerated container temperature
Temperature standard deviation, σ_T	3 °C	Random temperature fluctuation around the mean
Temperature threshold	8 °C	Alarm limit for unsafe temperature
Humidity mean, μ_H	40%	Average container humidity level
Humidity standard deviation, σ_H	3%	Random humidity fluctuation around the mean
Humidity threshold	55%	Alarm limit for unsafe humidity
Distribution type	Normal distribution	Used for both temperature and humidity readings

Each sensor node periodically generates environmental values. Temperature and humidity readings are generated using random distributions. Simulation input modelling requires selecting appropriate probability distributions to realistically

represent system randomness [11]. If a generated value exceeds the configured threshold, the application records an alarm event.

Network anomalies are commonly modeled as deviations from normal traffic behavior caused by failures, congestion, or abnormal communication patterns [4].The gateway is the main communication point between the sensor nodes and the backend server. It receives LoRa packets and forwards them. Queueing behaviour and packet loss probability are strongly affected by traffic variability and contention in communication networks [8]. A queueing point is considered at the gateway forwarding side, because packets arriving from multiple sensors may need to wait before being forwarded. In the current simulation results, the queue length stayed very low, which means that the gateway was not heavily overloaded under the selected configuration.

Table III gives the main entities, events, and state variables.

TABLE III
SIMULATION ENTITIES, EVENTS, AND STATE VARIABLES

Entity	Events	State Variables
Sensor Node	Sensor reading generation, packet transmission, alarm triggering	Temperature, humidity, alarm state, energy usage
LoRa Gateway	Packet reception, collision handling, packet forwarding	Received packets, collision count, queue length
Backend Server	Packet reception and monitoring	Received packet count, throughput
Wireless Channel	LoRa transmission and interference	Packet loss, contention state

Table IV shows the major modelling assumptions.

TABLE IV
MODELLING ASSUMPTIONS AND JUSTIFICATION

Assumption	Justification
Temperature is modelled using a Normal distribution	Temperature usually fluctuates around a mean value instead of being completely random
Humidity is modelled using a Normal distribution	Humidity values also vary around an expected level in a container
Sensor nodes report periodically	Wireless sensor networks commonly rely on periodic reporting for environmental monitoring applications [6]
Initial reporting offset is randomized	This helps avoid all sensors transmitting at the exact same time
LoRaWAN is used for wireless communication	LoRaWAN is suitable for low-power and long-range IoT systems
Gateway queue is monitored	The gateway is the main point where multiple sensor packets meet
Only three sensor nodes are used per container	This keeps the simulation manageable and easier to analyse
Simulation duration is 3600 seconds	One simulated hour gives enough events for basic statistical analysis

V. SIMULATION METHODOLOGY

The project is implemented as a discrete-event simulation. Discrete-event simulation is widely used for evaluating stochastic communication networks and packet-based systems under varying traffic and delay conditions [12]. The simulation does not update the full system continuously at every time step. Instead, it moves from one scheduled event to the next event. This is suitable for IoT communication systems because the main state changes happen when packets are generated, transmitted, received, or lost.

Random number generation is used in environmental value generation and repeated simulation runs. Reliable pseudo-random number generation is important in IoT simulations because stochastic events such as collisions, timer scheduling, and environmental variability depend on statistically valid random processes [7]. OMNeT++ built-in random variate generation and sampling functions were used to generate non-uniform random values from the selected distributions. Uniform random number generators are generally transformed into non-uniform distributions in order to generate random variables required by the simulation model [3].

Temperature and humidity values were generated using Normal distributions. Time-series-based stochastic modeling is commonly used in IoT sensor environments to simulate realistic temperature behavior and environmental fluctuation [5]. The random offset of reporting activity was used to reduce synchronized packet generation between nodes, since randomized timing can help reduce overlapping transmissions and medium access collisions [7].

TABLE V
SIMPLIFIED ENVIRONMENTAL READING AND EVENT GENERATION LOGIC

Pseudo Code
Generate sensor position using Uniform distribution
Generate reporting offset using Uniform distribution
Generate temperature using Normal(μ_T, σ_T)
Generate humidity using Normal(μ_H, σ_H)
If temperature $>$ threshold OR humidity $>$ threshold:
Trigger alarm event
Else:
Continue normal transmission
Transmit sensor packet through LoRaWAN

Monte Carlo simulation was used by repeating the simulation multiple times. In this project, 10 repeated runs were used. Each run uses a different random seed through the repeated run configuration. Different random seeds produce independent simulation replications, allowing statistically valid estimation of system performance metrics [10]. This allows the results to show not only one deterministic output but also the variability caused by random behaviour. A single simulation run is generally insufficient for reliable inference because stochastic variability may significantly affect the observed outputs [10].

Table VI shows the random processes and probability models used in the simulation.

TABLE VI
RANDOM PROCESSES AND PROBABILITY MODELS

Process	Distribution/Model	Purpose
Packet generation	Periodic timer with random offset	Generate routine sensor reports
Sensing event	Normal distribution	Generate temperature and humidity readings
Alarm event	Threshold-based stochastic event	Trigger alarm if value exceeds safe limit
Wireless collision	FLoRa wireless contention model	Represent LoRa interference and packet loss
Service/forwarding	Gateway forwarding process	Represent packet handling at gateway
Monte Carlo runs	Repeated runs with different seeds	Observe output variability

Table VII lists the scheduled events in the simulation.

TABLE VII
SCHEDULED EVENTS IN THE DISCRETE-EVENT SIMULATION

Scheduled Event	Trigger Condition	State Change / Action
Generate sensor reading	Sensor timer expires	Temperature and humidity values are generated
Transmit packet	Reporting interval reached	Sensor sends LoRa packet
Receive packet	Gateway receives signal	Packet count is updated
Collision event	Overlapping transmissions	Collision count increases and packet may be lost
Alarm generation	Reading exceeds threshold	Alarm signal is recorded
Gateway forwarding	Packet received at gateway	Packet is forwarded toward backend
End of run	Simulation reaches 3600 s	Scalars and vectors are recorded

VI. OMNeT++, INET, AND FLoRa IMPLEMENTATION

The project was implemented in OMNeT++ with INET and FLoRa integration. OMNeT++ provides the discrete-event simulation engine and module-based structure. OMNeT++ models are constructed from communicating modules that exchange messages through hierarchical network structures [9]. NED files are used to define the topology, while C++ files implement the application logic.

The main topology is defined in `ColdChainNetwork.ned`. This file connects the sensor nodes, gateway, and backend server. Sensor node behaviour is represented in `SensorNode.ned`, while gateway and backend structures are defined separately.

The main application behaviour is implemented in `ColdChainApp.cc` and `ColdChainApp.h`. This module is responsible for generating stochastic environmental values, creating packets, emitting signals, and checking alarm thresholds.

The simulation configuration is defined in `coldchain.ini`. This file includes the simulation duration, number of runs, sensor parameters, temperature and humidity parameters, LoRa settings, and output recording configuration.

Fig. 2 shows the OMNeT++ project structure.

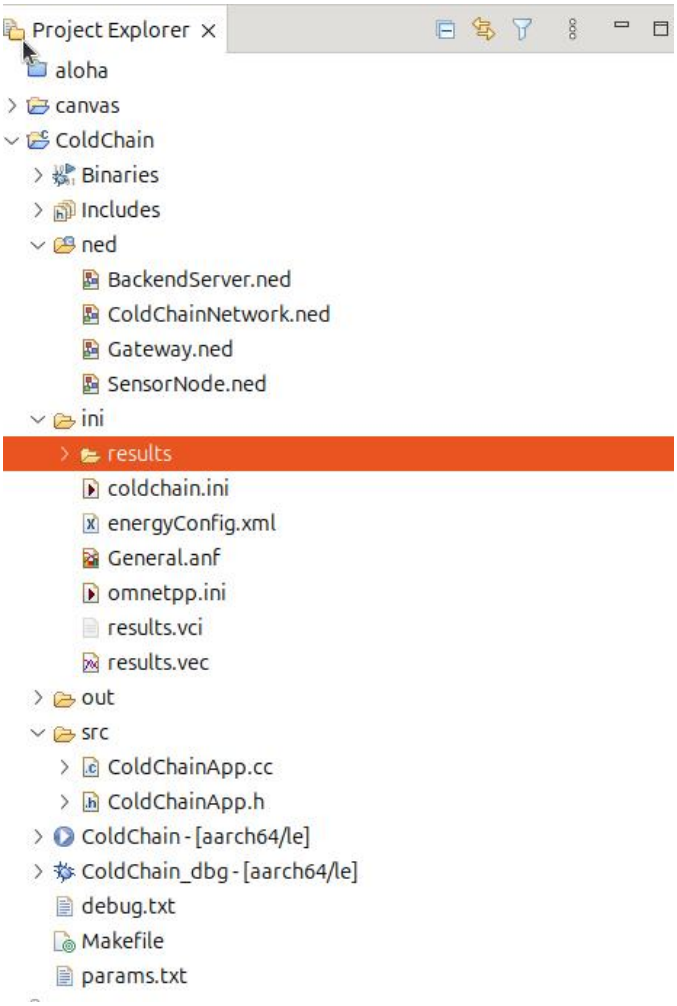

Fig. 2. OMNeT++ project structure and main implementation files.

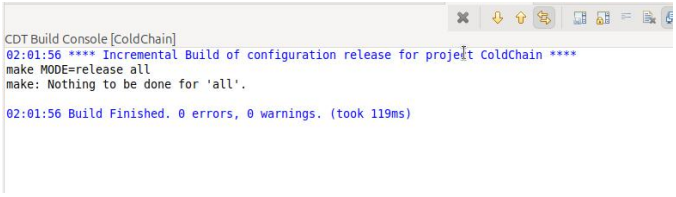

Fig. 3. Successful compilation of the simulation project in OMNeT++.

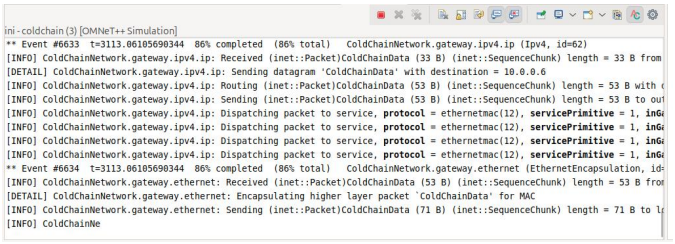

Fig. 4. Simulation runtime logs showing packet transmission and gateway forwarding events.

TABLE VIII
IMPLEMENTED OR MODIFIED PROJECT FILES

File Name	Type	Purpose
ColdChainNetwork.ned	NED	Defines complete simulation network topology
SensorNode.ned	NED	Defines the sensor node module
Gateway.ned	NED	Defines the LoRa gateway module
BackendServer.ned	NED	Defines backend monitoring server
ColdChainApp.cc	C++	Implements sensor application logic
ColdChainApp.h	Header	Declares application variables and functions
coldchain.ini	INI	Contains simulation configuration and parameters
energyConfig.xml	XML	Defines energy-related configuration
General-*.sca	Result	Scalar output files from repeated runs
General-*.vec	Result	Vector output files from repeated runs

Table IX explains the use of OMNeT++, INET, and FLoRa.

TABLE IX
OMNET++, INET, AND FLORA USAGE

Tool/Framework	How It Is Used in the Project
OMNeT++	Main discrete-event simulation platform. It schedules events, runs modules, and records scalar/vector outputs.
INET	Backend network representation and IP/Ethernet-related components.
FLoRa	LoRaWAN communication, gateway reception, wireless interference, and LoRa packet behaviour. FLoRa is widely used in OMNeT++ environments for evaluating LoRaWAN and IoT communication scenarios [6].

VII. MAPPING OF COURSE TOPICS TO THE PROJECT

Table X maps the CNG 476 course topics to the implemented project.

VIII. EXPERIMENTAL DESIGN

The experiments were performed using 10 repeated simulation runs in order to make sure that the steady-state is achieved. Each run lasted 3600 seconds. The same topology was used, but each run produced different random behaviour due to random number generation and seed variation.

The baseline scenario uses the standard reporting configuration and three sensor nodes. The second scenario is interpreted from higher-contention repeated runs where packet collisions increased and packet delivery performance decreased. This comparison is useful because wireless contention is one of the main factors affecting LoRaWAN reliability.

Table XI summarizes the experimental settings.

TABLE XI
EXPERIMENTAL DESIGN SUMMARY

Item	Value / Description
Simulation Duration	3600 seconds
Number of Runs	10 repeated runs
Random Seeds	Different seed per repeated run
Baseline Scenario	Standard reporting and normal contention
Higher Contention Scenario	Increased overlapping wireless transmissions and collisions
Alternative Scenario	Higher contention runs with increased collisions
Varied Parameters	Random environmental readings and wireless behaviour
Collected Metrics	PDR, collisions, energy, alarms, queue length, throughput

IX. PERFORMANCE METRICS

The performance metrics were selected to evaluate both the application-level monitoring behaviour and network-level communication performance.

Packet Delivery Ratio (PDR) is the main reliability metric. PDR is one of the most commonly used performance metrics in LoRaWAN and wireless sensor network simulations [6]. Collision count is used to evaluate the effect of wireless contention. Energy consumption is important because IoT sensor nodes are usually battery-powered. Alarm count shows how often unsafe environmental readings were detected. Queue length is used to observe whether packets accumulate at the gateway.

The Packet Delivery Ratio is calculated as:

$$PDR = \frac{\text{Successfully Received Packets}}{\text{Transmitted Packets}} \times 100 \qquad (1)$$

Table XII defines the performance metrics.

TABLE X
MAPPING OF CNG 476 COURSE TOPICS TO THE FINAL PROJECT

Course Topic	Project-Specific Implementation
Simulation methodology and modelling	Entities, events, state variables, assumptions, and output metrics were defined.
Random number generation	Pseudo-random values generate stochastic sensor readings and repeated simulation runs [7].
Monte Carlo simulation	The simulation is repeated 10 times with different seeds to observe stochastic variability [10].
Probability models	Normal distributions model stochastic environmental behaviour [11].
Inverse transformation or sampling	OMNeT++ sampling utilities generate non-uniform values.
Stochastic processes	Sensor readings, alarms, and collisions behave stochastically.
Queueing analysis	Gateway forwarding is monitored as the queueing point.
Discrete-event simulation	Events change the system state at specific times.
Event scheduling	Sensor timers schedule reporting and sensing events.
Time-stepped / trace-driven view	The model is DES-based; vectors provide time-based traces.
Wireless network modelling	FLoRa models LoRa nodes, gateway access, and interference.
OMNeT++ implementation	NED, C++, INI, scalar, and vector files are used.
INET integration	INET supports backend communication structure.
FLoRa integration	FLoRa provides LoRa communication and gateway behaviour.
Input analysis	Histograms and vectors analyse sensor readings.
Output analysis	Averages, variability, plots, and confidence intervals analyse outputs.
IoT application relevance	The project represents cold-chain IoT monitoring.

TABLE XII
PERFORMANCE METRICS

Metric	Unit	Definition / Purpose
Packet Delivery Ratio	%	Ratio of successfully received packets to transmitted packets
Collision Count	packets	Number of packet collisions observed at gateway
Queue Length	packets	Number of packets waiting at gateway forwarding point
Throughput	packets/s	Successfully received packets per second
Energy Consumption	J	Energy consumed by sensor nodes
Alarm Count	alarms	Number of threshold violation events
Temperature	°C	Generated environmental temperature value
Humidity	%	Generated humidity value inside the monitored container

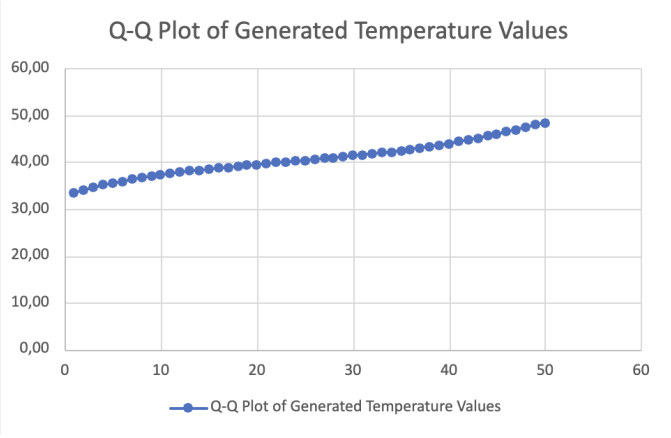

Fig. 5. Q-Q plot of generated temperature values against the theoretical Normal distribution.

X. INPUT ANALYSIS

Input analysis was performed mainly on the generated environmental readings. The temperature and humidity values were generated using Normal distributions. In the implementation, the sensor node application samples these values during packet generation events. Simulation models proceed by generating random samples from specified probability distributions that represent system uncertainty [11].

Q-Q (Quantile-Quantile) plots were generated for both temperature and humidity values in order to compare the sampled outputs against the theoretical Normal distribution. In both cases, most points approximately follow the expected trend, indicating that the generated environmental readings are reasonably consistent with the assumed Normal distribution used in the simulation model.

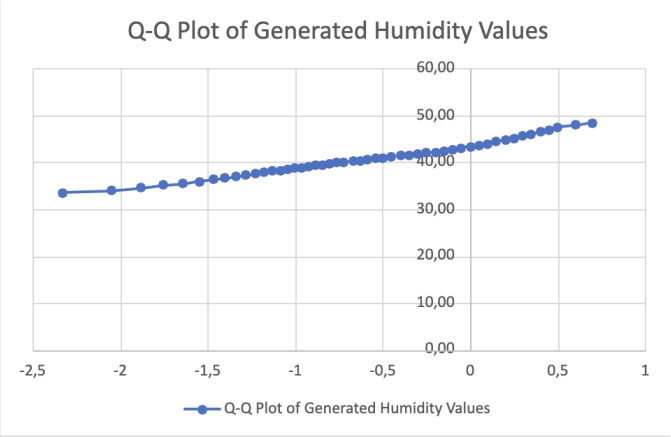

Fig. 6. Q-Q plot of generated humidity values against the theoretical Normal distribution.

The histogram of temperature values approximately follows a bell-shaped form. Histograms are commonly used to visually

evaluate whether a selected probability distribution appropriately represents simulation input data [11]. This supports the assumption that the generated temperature values are consistent with a Normal distribution. The time-series plot also shows that values fluctuate randomly around the configured mean instead of following a fixed deterministic pattern.

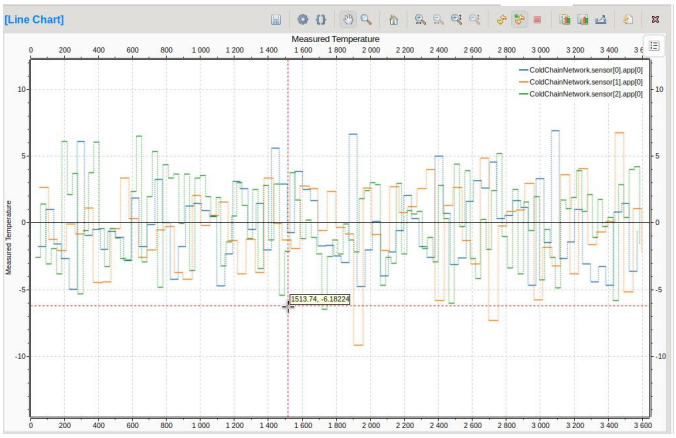

Fig. 7. Temperature variation over simulation time for the sensor nodes.

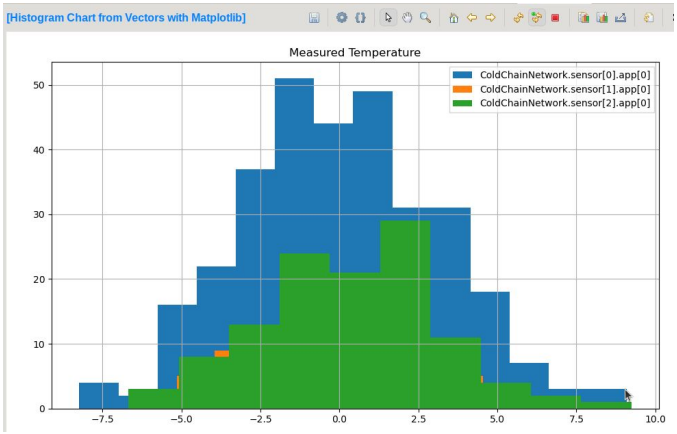

Fig. 8. Distribution of generated temperature values collected from sensor nodes during repeated simulation runs.

Fig. 9 shows the generated humidity values over the simulation time. The readings fluctuate around the configured mean humidity value and show that the humidity generation component is active.

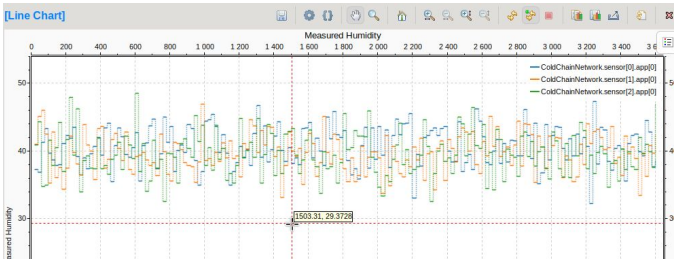

Fig. 9. Humidity variation over simulation time for the sensor nodes.

TABLE XIII
INPUT ANALYSIS SUMMARY

Input Variable	Analysis Method	Conclusion
Temperature	Histogram and time-series plot	Approximately follows Normal variation
Humidity	Time-series plot	Fluctuates around configured mean
Alarm/Fault Event	Threshold model	Alarm occurs when safe value is exceeded
Packet Generation	Timer-based model	Periodic reporting with random offset

XI. RESULTS AND OUTPUT ANALYSIS

The output analysis was performed using scalar and vector files generated by OMNeT++. The main analysed metrics were Packet Delivery Ratio, collisions, energy consumption, and alarm count.

Across 10 repeated runs, the average Packet Delivery Ratio was approximately 81.64%. This means that most of the generated packets were successfully received. However, not all packets were delivered because LoRa wireless communication is affected by interference and collisions. Previous studies also show that communication contention and interference reduce packet delivery performance in LoRaWAN systems [6].

The 95% confidence interval for the Packet Delivery Ratio was approximately $\pm 3.95\%$. The confidence interval was estimated using the sample mean and standard deviation obtained from the 10 repeated simulation runs. This is useful because one simulation run alone may not represent the whole stochastic behaviour of the system. Using multiple independent replications improves the statistical reliability of simulation-based performance evaluation [10].

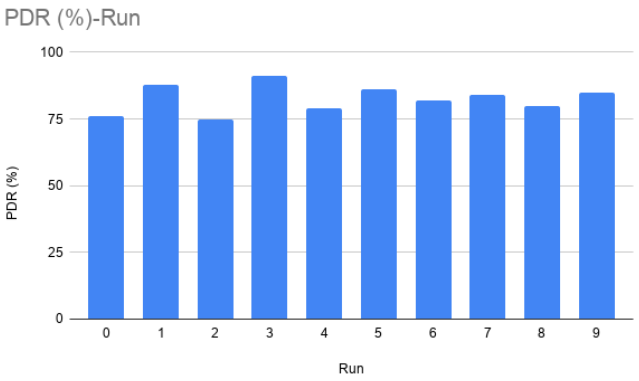

Fig. 10. Packet Delivery Ratio across Monte Carlo simulation runs.

The average collision count was approximately 86.1 collisions per run. Runs with higher collision counts generally had lower packet delivery performance. This is expected because overlapping LoRa transmissions make it harder for the gateway to decode packets correctly. Collisions mainly occurred when multiple sensor nodes attempted transmission

within overlapping time intervals. Communication network simulations frequently model packet forwarding behaviour using FIFO queueing disciplines and stochastic service delays [12].

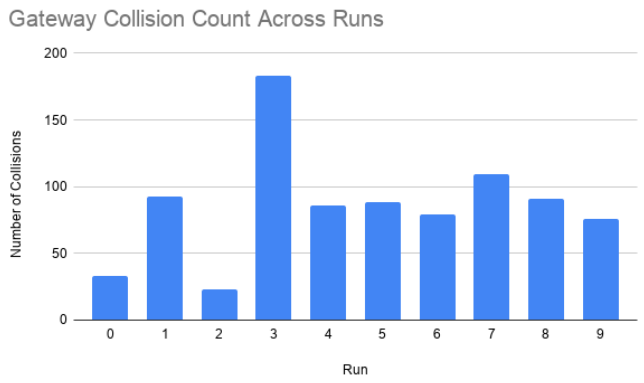

Fig. 11. Gateway collision counts observed across repeated simulation runs.

Energy consumption was also recorded. The average sensor energy consumption was approximately 28.86 J. Since sensor nodes in cold-chain monitoring are usually battery-powered, this metric is important for longer deployments.

Queue statistics were also monitored at the gateway. The measured queue length stayed close to zero in the uploaded results. This means that the implemented workload did not heavily overload the gateway forwarding process. However, the gateway is still a valid queueing point because a larger number of sensor nodes or a smaller reporting interval could create packet accumulation. Previous studies also show that queueing behaviour and packet loss strongly depend on traffic characteristics and network load conditions [8].

Table XIV summarizes the main simulation results.

TABLE XIV
SIMULATION RESULTS SUMMARY

Scenario	Metric	Average	Std. Dev. / CI
Baseline	Packet Delivery Ratio	81.64%	±3.95% CI
Baseline	Collision Count	86.1	Variable by run
Baseline	Energy Consumption	28.86 J	Moderate variation
Baseline	Alarm Count	1.5	Low frequency
Higher contention	PDR	Lower than average	Caused by collisions
Higher contention	Collision Count	Higher than average	More wireless overlap

XII. DISCUSSION

The results show that the implemented LoRaWAN-based cold-chain monitoring system can deliver most environmental sensor packets successfully. The average PDR was above 80%, which is acceptable for a basic monitoring simulation. However, the results also show that wireless collisions affect packet delivery.

The collision behaviour is important because LoRaWAN networks are usually shared wireless environments. If many sensors transmit at similar times, the gateway may receive overlapping signals. In this project, the number of sensor nodes was only three, but collisions were still observed. With more sensor nodes, it is expected that contention would increase.

The temperature and humidity generation process demonstrates stochastic environmental modelling. Instead of using fixed deterministic readings, the sensor values changed randomly around a mean. This is more realistic than using constant values, because real cold-chain containers also experience environmental fluctuation. Temperature fluctuations during transportation can significantly affect the safety and quality of temperature-sensitive products, making continuous monitoring important in cold-chain systems [15].

The queueing results were weaker than expected because the gateway queue stayed almost empty. This means that the current system was not overloaded enough to create visible queue buildup. Still, the queueing point is technically meaningful because gateway forwarding can become a bottleneck in larger topologies.

Table XV summarizes the main findings.

TABLE XV
DISCUSSION OF MAIN FINDINGS

Finding	Explanation
PDR stayed above 80%	The LoRaWAN system delivered most packets successfully
Collisions affected delivery	Higher collision values reduced communication reliability
Environmental values were stochastic	Temperature and humidity were generated from probability distributions
Queue length stayed low	The selected workload did not heavily overload the gateway
Energy was measurable	Energy consumption was recorded for sensor nodes

The main limitation is that the implemented topology contains only three sensor nodes. This makes the simulation easier to run and analyse, but it does not fully represent a large logistics deployment. Future work can increase the number of sensor nodes, add realistic mobility traces, and test adaptive reporting intervals.

XIII. CONCLUSION

This project implemented a cold-chain logistics monitoring system using OMNeT++, INET, and FLoRa. The system contains sensor nodes that generate stochastic environmental readings and transmit them through a LoRa gateway to a backend server.

The project demonstrates several system simulation concepts, including discrete-event simulation, random number generation, probability models, stochastic processes, Monte Carlo repeated runs, queueing analysis, input analysis, output analysis, and IoT wireless network modelling.

The results show that the simulated system achieved an average Packet Delivery Ratio of approximately 81.64%. Collisions were observed and affected packet delivery performance. Environmental values followed the expected stochastic behaviour, and threshold-based alarms were generated during the simulation.

The main limitation is the small network size. A future extension can include more sensor nodes, more complex mobility, stronger gateway queueing analysis, and adaptive alarm reporting policies.

Code Availability

The complete working project source code, simulation files, configuration files, result files, screenshots, final report PDF, and result-processing files are publicly available at:

https://github.com/irem0410/COLDCHAIN-MONITORING.git

Individual Contributions

TABLE XVI
Individual Contributions

Name	Student ID	Contribution
Ali Murat Chatkha	2486843	OMNeT++ implementation, FLoRa integration, project configuration, simulation execution, statistical analysis, report writing, result interpretation, input/output analysis
Irem Dede	2584910	OMNeT++ implementation, FLoRa integration, project configuration, simulation execution, statistical analysis, report writing, result interpretation, input/output analysis

References

[1] I. F. Akyildiz, W. Su, Y. Sankarasubramaniam, and E. Cayirci, "Wireless sensor networks: a survey," *Computer Networks*, vol. 38, no. 4, pp. 393–422, 2002.

[2] T. T. Baladraf, "Integrated Vehicle Routing and Cold Chain Optimization with Seasonal Variability Simulation for Fresh Fruit Distribution Networks," *JUTIN: Jurnal Teknik Industri Terintegrasi*, vol. 8, no. 4, pp. 4280–4291, 2025.

[3] P. L'Ecuyer, "Random Number Generation," 2025.

[4] M. Thottan and C. Ji, "Anomaly Detection in IP Networks," *IEEE Transactions on Signal Processing*, vol. 51, no. 8, pp. 2191–2204, 2003.

[5] L. V. Vigoya, V. Casallas, D. F. Castellanos, and F. Correa, "DAD: A Labeled IoT Dataset for Anomaly Detection and Statistical Sensor Modeling," *Sensors*, vol. 20, no. 13, 2020.

[6] S. Idris, T. Karunathilake, and A. Förster, "Survey and Comparative Study of LoRa-Enabled Simulators for Internet of Things and Wireless Sensor Networks," *Sensors*, vol. 22, no. 15, p. 5546, 2022.

[7] P. Kietzmann, T. C. Schmidt, and M. Wählisch, "A Guideline on Pseudorandom Number Generation in the IoT," *ACM Computing Surveys*, vol. 54, no. 6, pp. 112:1–112:35, 2021.

[8] K. Rukkas, A. Morozova, I. Meniailov, and M. Momot, "Analysis of Packet Loss Probability Models in a Router Buffer Based on Traffic Fractality," *Radioelectronic and Computer Systems*, 2025.

[9] A. Varga and R. Hornig, "An Overview of the OMNeT++ Simulation Environment," in *Proceedings of SIMUTools*, 2008.

[10] A. M. Law, "Statistical Analysis of Simulation Output Data," in *Proceedings of the Winter Simulation Conference*, 2020.

[11] A. M. Law, "How to Build Valid and Credible Simulation Models," in *Proceedings of the Winter Simulation Conference*, 2011.

[12] Y. [Surname], "Modeling and Simulation of Transport Coding in Communication Networks," 2025.

[13] Q. Ma, B. Xie, P. Cao, X. Xie, J. Li, Y. He, and C. Li, "Emerging Ternary Eutectic Hydrated Salt Cold Energy Storage Materials for Cold Chain Transportation," *Energy*, 2025.

[14] K. Mounika et al., "Post-Harvest Technologies and Cold Chain Management: A Review," *Plant Archives*, vol. 25, 2025.

[15] M. S. Yar, I. H. Ibeogu, A. Regmi, N. Zhang, and C. Li, "Advances in Intelligent Time-Temperature Indicators for Cold Chain Monitoring," *Trends in Food Science & Technology*, 2025.